

CommentCountCheck

Tests:

- ★ testCommentNodesRequired()
 - Used to verify that the check included comment nodes in it's construction of the AST
- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - The test case for each of these was just ensuring that the only successful return value for each get_Tokens() method was the expected array
- ★ visitToken()
 - Checked that the initialization of the values reset them
 - In this class, checked the cases in which visitToken() was run 1 and 5 times
- ★ beginTree()
 - Checked that beginning a tree reset the class variables
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected value, after running visitToken() 3 times
- ★ testGet()
 - Tested that the get method for the comment count returned the correct values, at initialization.
 - This was only tested at initialization, because in the finishTree() test, the method was used/tested for.

Test coverage:

Overall coverage: 52.8%

Line coverage: 53.8%

Branch coverage: 0/0 branches covered

CommentLineCountCheck

Tests:

- ★ testCommentNodesRequired()
 - Used to verify that the check included comment nodes in it's construction of it's AST
- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - The test case for each of these was just ensuring that the only successful return value for each get_Tokens() method was the expected array
- ★ visitToken()
 - Checked that the initialization of the values reset them
 - In this class, checked the cases in which visitToken() was run once with a SINGLE_LINE_COMMENT, and correctly found the values for a multi-line comment starting on line 2 and ending on line 10

- ★ beginTree()
 - Checked that beginning a tree reset the class variables/kept them at 0
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected value, after running visitToken() 3 times
- ★ testGet()
 - Tested that the get method for the comment line count returned the correct values, at initialization, and after running visitToken() with a SINGLE_LINE_COMMENT
 - Also, in a separate test, tested that the second get method returned correctly when getting a multi-line comment starting on 2

Test coverage:

Overall coverage: 33.7%

Line coverage: 33.3%

Branch coverage: 3t 3f/3 branches covered

ExpressionCountCheck

Tests:

- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - The test case for each of these was just ensuring that the only successful return value for each get_Tokens() method was the expected array
- ★ visitToken()
 - Checked that the initialization of the values reset them
 - In this class, checked the cases in which visitToken() was run three times, where the next sibling (AKA another node on the same line) was not null, and when visitToken() was run three times without the next sibling being null
- ★ beginTree()
 - Checked that beginning a tree reset the class variables/kept them at 0
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected value, after running visitToken() once, with the next sibling as null
- ★ testGet()
 - Tested that the get method for the expression count returned the correct values, at initialization, and after running visitToken() 5 times

Test coverage:

Overall coverage: 47.3%

Line coverage: 33.3%

Branch coverage: 1t 1f/1 branches covered

LoopCountCheck

Tests:

- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - The test case for each of these was just ensuring that the only successful return value for each get_Tokens() method was the expected array
- ★ visitToken()
 - Checked that the initialization of the values reset them
 - Checked that the count updated after running visitToken() once
- ★ beginTree()
 - Checked that beginning a tree reset the class variables/kept them at 0
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected value, after running visitToken() 3 times
- ★ testGet()
 - Tested that the get method for the loop count returned the correct values, at initialization, and after running visitToken() 5 times

Test coverage:

Overall coverage: 55.6%

Line coverage: 41.7%

Branch coverage: 0/0 branches covered

OperandCountCheck

Tests:

- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - The test case for each of these was just ensuring that the only successful return value for each get_Tokens() method was the expected array
- ★ visitToken()
 - Checked that the initialization of the values reset them
 - Checked that the general count updated after running visitToken() twice with “true”, and that the unique count only updated once, and also that the general count updated with “false”, but that the unique count only increased once— all in the same test
- ★ beginTree()
 - Checked that beginning a tree reset the class variables/kept them at 0, and that running visitToken() once updated the counts
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected values, for both the general and unique counts, after running visitToken() 3 times
- ★ testGet()/testSet()

- Tested that the set methods properly updated the counts, when one was set to 0 and the other 100000000, then with one being 16 and the other as -2

Test coverage:

Overall coverage: 40.1%

Line coverage: 41.9%

Branch coverage: 2t 2f/2 branches covered

OperatorCountCheck

Tests:

- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - The test case for each of these was just ensuring that the only successful return value for each get_Tokens() method was the expected array
- ★ visitToken()
 - Checked that the initialization of the values reset them
 - Checked that the general count updated after running visitToken() three times with "==", and that the unique count only updated once, and also that the general count updated with "/", but that the unique count only increased once– all in the same test
- ★ beginTree()
 - Checked that beginning a tree reset the class variables/kept them at 0, and that running visitToken() twice updated both the general and unique counts
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected values, for both the general and unique counts, after running visitToken() 3 times
- ★ testGet()/testSet()
 - Tested that the set methods properly updated the counts, when one was set to 0 and the other 100000000, then with one being 16 and the other as -2 (these were the same as the operand count tests :)

Test coverage:

Overall coverage: 78.3%

Line coverage: 46.7%

Branch coverage: 3t 3f/3 branches covered

HalsteadLengthCheck

Tests:

- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()

- As a Halstead check, these methods had to be initialized, and the test here was that the return value was an array of {0}
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected value (0), after setting the operand and operator counts (all 0)
- ★ testGet()
 - Tested that the get method for the Halstead effort calculation returned the correct value (2, with the operator count as negative), after setting the operand and operator counts

Test coverage:

Overall coverage: 61.8%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered

HalsteadVocabularyCheck

Tests:

- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - As a Halstead check, these methods had to be initialized, and the test here was that the return value was an array of {0}
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected value (0), after setting the operand and operator counts (both 0)
- ★ testGet()
 - Tested that the get method for the Halstead effort calculation returned the correct value (3, with a negative operand count), after setting the operand and operator counts

Test coverage:

Overall coverage: 61.8%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered

HalsteadDifficultyCheck

Tests:

- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - As a Halstead check, these methods had to be initialized, and the test here was that the return value was an array of {0}
- ★ finishTree()

- Used the get method to find if the value retrieved for the log function matched the expected value, after setting the operand and operator counts
- ★ testGet()
 - Tested that the get method for the Halstead effort calculation returned the correct value (3 and 0 (as separate tests), without negative values), after setting the operand and operator counts

Test coverage:

Overall coverage: 67.5%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered

HalsteadVolumeCheck

Tests:

- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - As a Halstead check, these methods had to be initialized, and the test here was that the return value was an array of {0}
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected value, after setting the operand and operator counts
- ★ testGet()
 - Tested that the get method for the Halstead volume calculation returned the correct value (8), after setting the operand and operator counts

Test coverage:

Overall coverage: 69.8%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered

HalsteadEffortCheck

Tests:

- ★ getAcceptableTokens(); getDefaultTokens(); getRequiredTokens()
 - As a Halstead check, these methods had to be initialized, and the test here was that the return value was an array of {0}
- ★ finishTree()
 - Used the get method to find if the value retrieved for the log function matched the expected value, after setting the operand and operator counts
- ★ testGet()
 - Tested that the get method for the Halstead effort calculation returned the correct value (16, in this case), after setting the operand and operator counts

Test coverage:

Overall coverage: 60.6%

Line coverage: 71.4%

Branch coverage: 0/0 branches covered